



GraphView

Your documents, finally connected.

IN THREE SENTENCES

GraphView ingests your documents — PDFs, Office files, source trees, whole dropped folders, or just a URL — and lays them out as a living force-directed *atlas* of documents, the concepts inside them, your notes and AI output, connected by typed, machine-checked edges. A deliberately lightweight, cost-metered AI assistant sits on top: it summarises, extracts concepts, and answers questions across everything you have read, with citations verified against real nodes instead of invented. Your files stay yours — on desktop the primary store is an ordinary folder on your own disk, laid out the way the graph is, readable without the app and exportable at any time.

1,449

automated tests

142

API operations

9

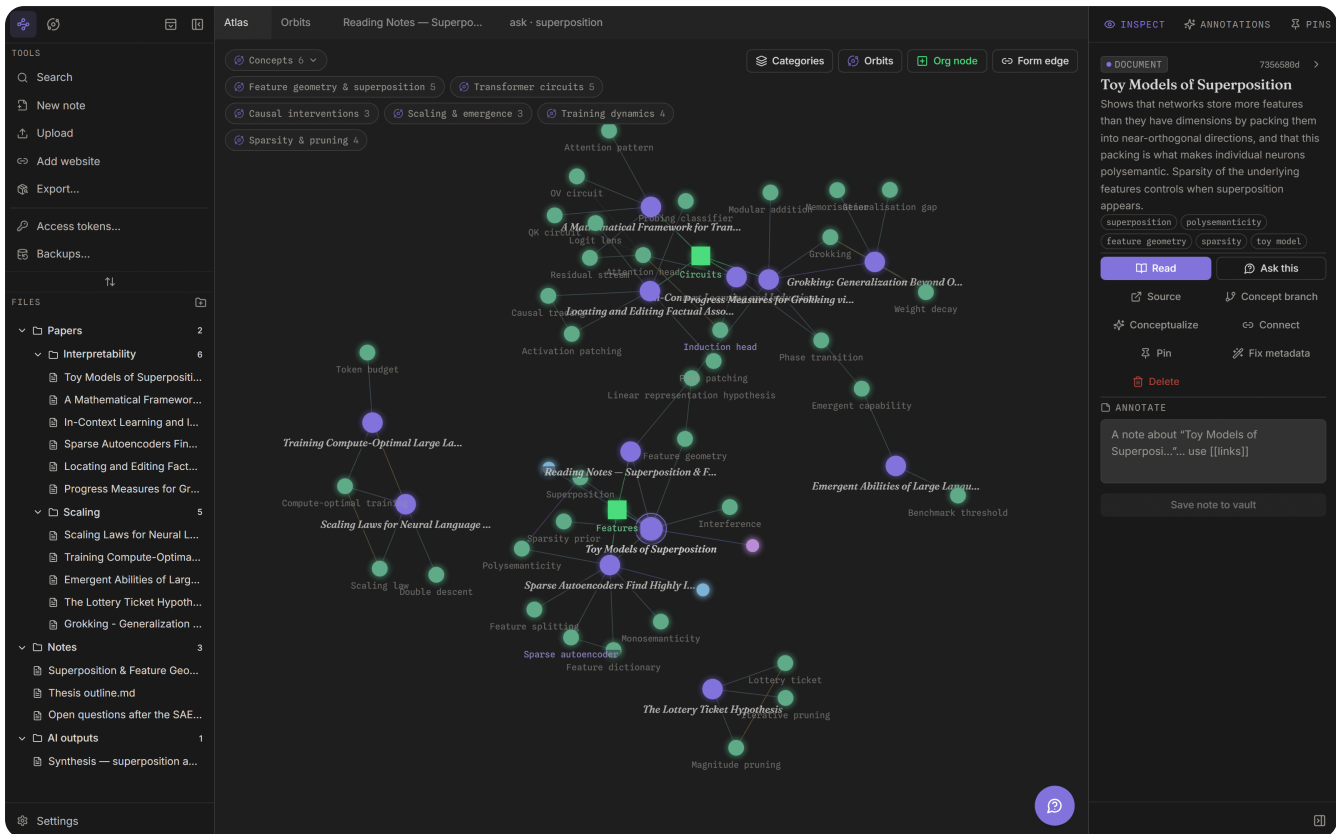
CI jobs per push

8

decision records

A graph-native personal knowledge system

graphview.org



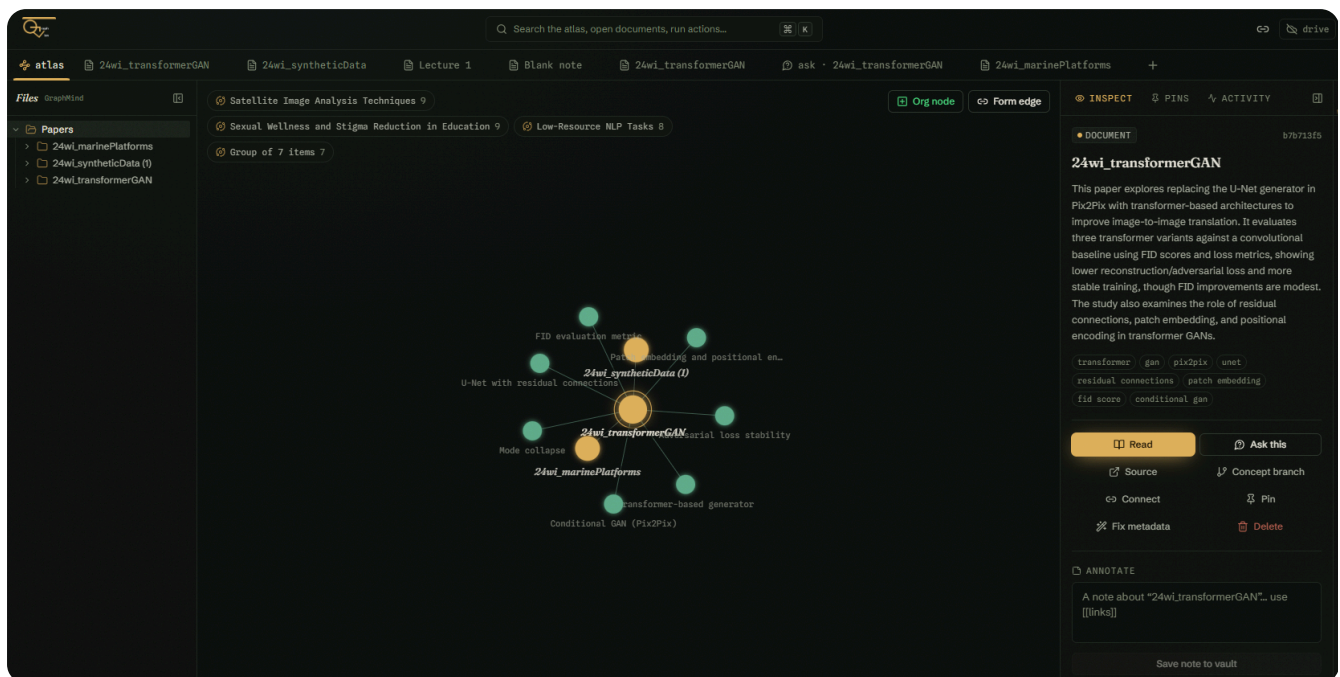
The studio. Your files on the left, the atlas in the middle, the inspector on the right.

The whole idea, in two pictures

You have a folder. It has everything you have ever read in it, and it tells you nothing.

	Contact Lens Rx	6/11/2026 4:23 AM	PNG File	60 KB
	24wi_transformerGAN	6/7/2026 2:10 AM	Chrome PDF Docum...	3,936 KB
	24wi_marinePlatforms	6/7/2026 2:09 AM	Chrome PDF Docum...	383 KB
	24wi_syntheticData (1)	6/7/2026 2:09 AM	Chrome PDF Docum...	682 KB
	24wi_syntheticData	6/7/2026 2:09 AM	Chrome PDF Docum...	682 KB
	DSAPractice	6/13/2026 5:43 PM	File folder	

GraphView parses, embeds and connects the same material — sections, concepts, and the relationships between them — and hands it back as something you can actually think in.



The same folder, as an atlas. (The studio ships several themes; this one is Brass.)

“NotebookLM is document-centric Q&A. Obsidian is manual-note-centric. This is knowledge-centric: automatic structure over everything you’ve ever read, with the files left where they already live.”

What you can actually do with it

Put things in. Drag a file — or an entire folder, hierarchy preserved. PDFs, `.docx`, `.pptx`, HTML, Markdown, plain text, and around forty source and config formats. Paste a URL and the page is fetched, read, snapshotted to PDF and ingested. Connect Google Drive if you want to; you do not have to.

Watch it become structure. Every ingest runs four stages — parse, structure, enrich, embed — each committed separately, so the progress you see is real progress and not a spinner. A failed AI stage does not fail the ingest.

What happens to a file you drop in

Drag a PDF — or a whole folder, a .docx, a slide deck, a source tree, or just a URL. Four stages, each one committed separately so you watch it happen.

Parse

DETERMINISTIC

pypdf on the fast path; Docling when the layout matters. Text, sections, figures, tables, arXiv metadata, bibliography.

Structure

DETERMINISTIC

Nodes and typed edges, no model involved. The same file always yields the same skeleton — and it is ontology-checked.

Enrich

LLM · NON-FATAL

A model writes summaries and proposes concepts. Cached by content hash; if it fails, the ingest still succeeds without it.

Embed

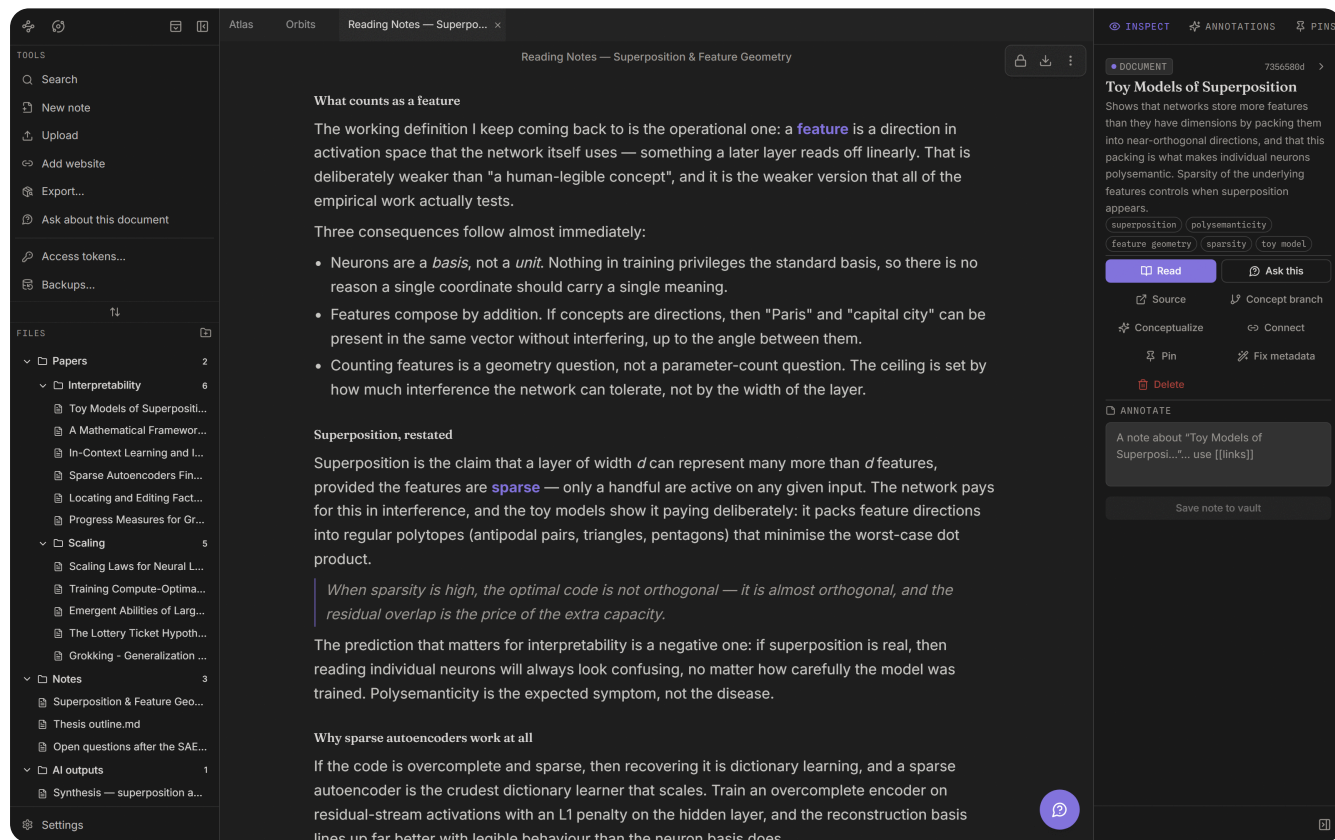
LOCAL · NON-FATAL

Vectors for semantic search, computed by an ONNX embedder running on your own machine.

And then it is on the atlas.

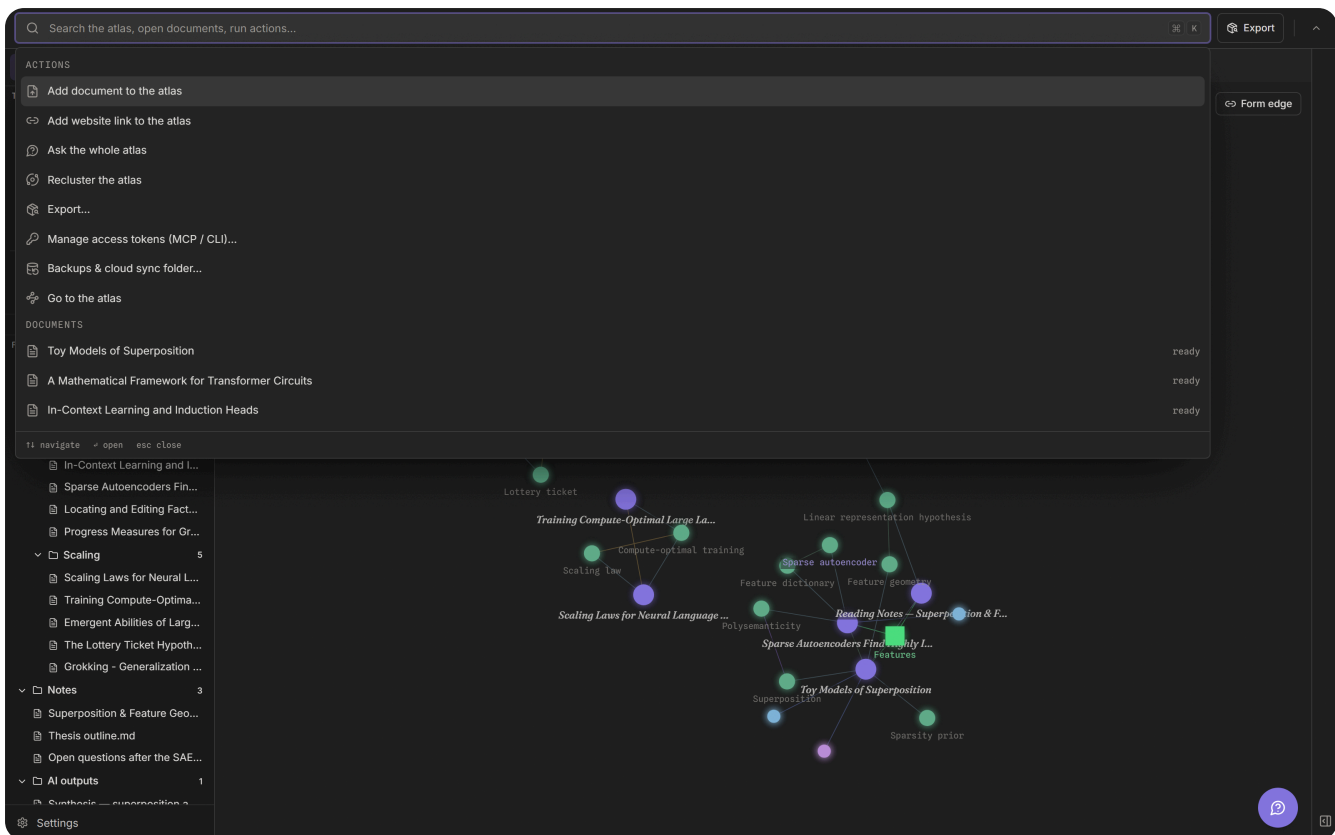
Findable by keyword and by meaning at once (FTS5 bm25 u vector cosine, fused). Askable, with citations verified against real node ids. Linked to everything related

Read, mark up, and write. A real PDF reader with handwriting — pen, highlighter, eraser, text, stylus pressure, palm rejection — an infinite canvas for the things that do not fit on a page, and OCR that turns your ink into searchable text on the node. For prose there is an Obsidian-style Markdown editor on CodeMirror 6 with Live Preview, `[[wikilinks]]` that resolve against the graph, and KaTeX math. Every panel docks, floats and snaps, and the workspace remembers how you left it.

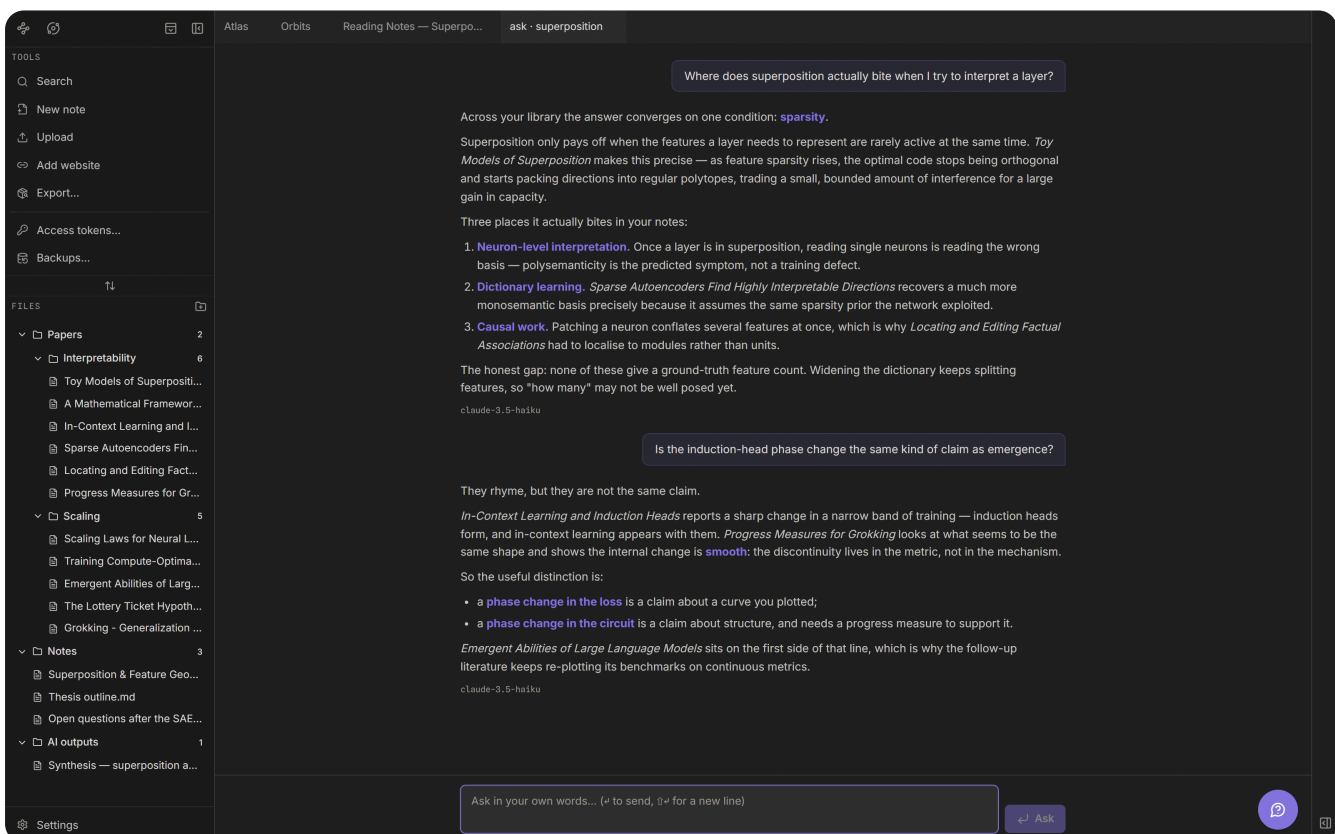


A note open in the reader, with the source document inspected beside it.

Find things two ways at once. Lexical search (SQLite FTS5/bm25, or Postgres tsvector when hosted) and semantic search (vector cosine) run together and are fused by reciprocal rank — so “the paper about the thing with the residual connections” works as well as an exact phrase. One palette searches the atlas, opens documents and runs actions.

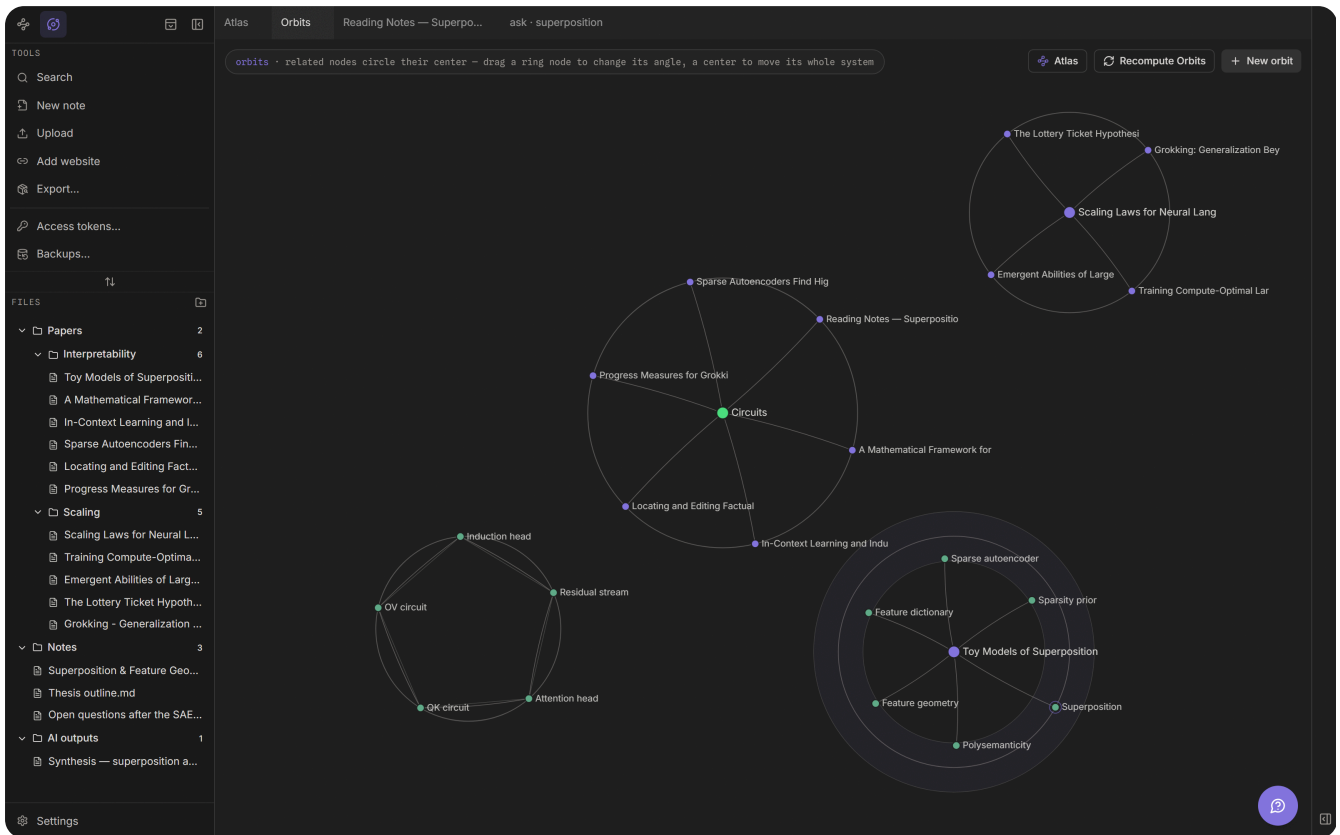


Ask across all of it. Select a passage and ask for an explanation, in five modes, streamed. Or ask the whole atlas a question and get an answer whose citations have been checked against real node ids — invented ones are dropped and counted.



Answers name the documents they came from, and the model that produced them is on the record.

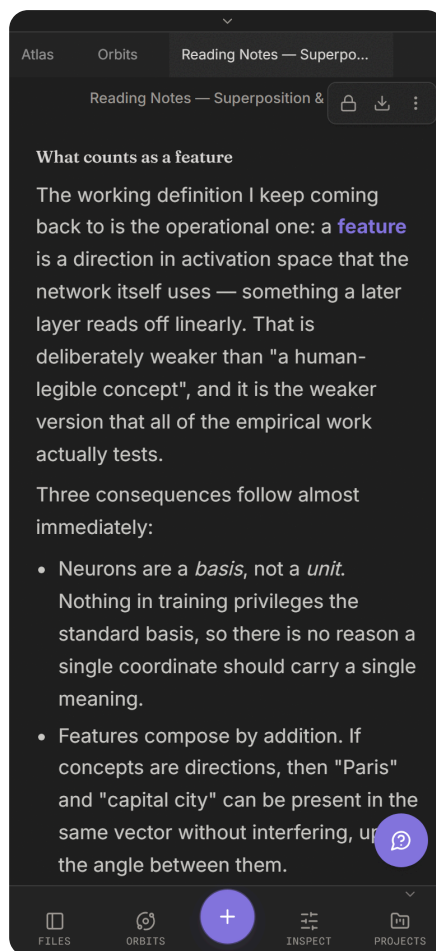
See it a second way. Orbits is a different projection of the same graph: hubs at the centre, related material seated on rings around them, in four orbit shapes. Useful when a force layout has told you everything it is going to.



Take it with you. Nine export formats, including an Obsidian-ready vault zip, a vault-shaped Markdown tree with wikilinks intact, and a Graphiti-format agent bundle. Full-account export, and account deletion that actually deletes.

Plug your knowledge into your other tools. A published MCP server (`uvx graphmind-mcp`) exposes search, read, ingest, note-writing and grounded ask to Claude, Cursor, or any MCP client — authenticated with a scoped personal access token you mint in the app.

Where it runs



The same studio on a phone — one pane, a bottom dock, and the reader running full width.

SURFACE	STATUS
Web studio — <code>app.graphview.org</code> , responsive down to a phone	shipped
Installable PWA — desktop, tablet and phone, no app store	shipped
Windows desktop app — one self-contained process serving the studio and API entirely on-device	shipped, installer smoke-built on every push
MCP server — published to PyPI as <code>graphmind-mcp</code>	shipped
Atlas site API — scoped, per-document opt-in read + chat, for putting your knowledge behind a public site	shipped
Tauri desktop shell — 3-OS release matrix incl. universal macOS	built, not yet the distributed path

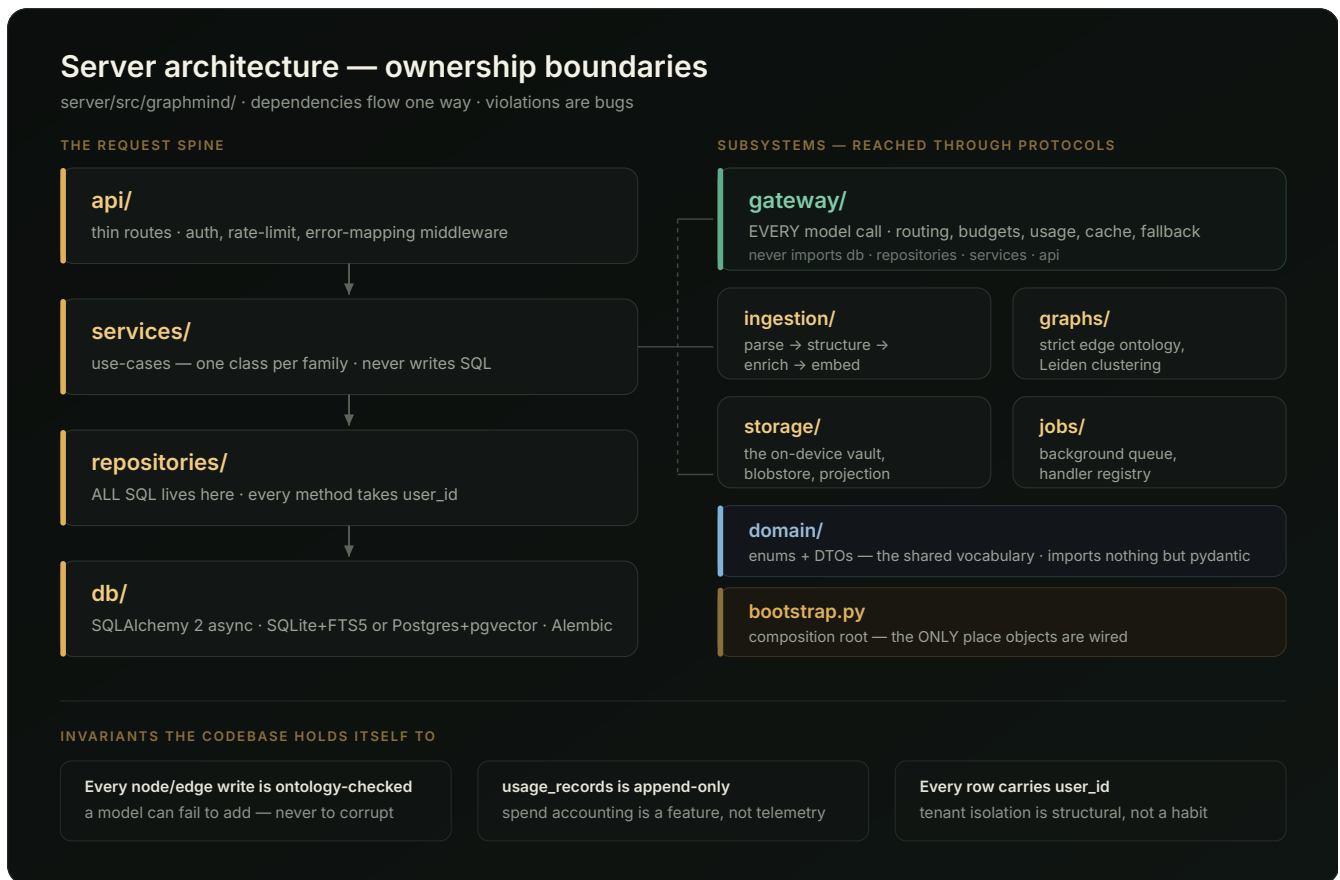
The engineering

This section is the point of this document. Everything below is a count taken from the tree, not a claim.

By the numbers

1,449	automated tests — 929 backend (pytest), 441 frontend unit (vitest), 79 end-to-end (Playwright)
3	device profiles every E2E case runs against — desktop Chrome, Pixel 5, and a touch tablet at 1194×834
21,763	lines of test code, against 31,418 lines of backend source — a 0.69 : 1 ratio
9	CI jobs on every push, including a Windows lane and a Postgres-with-pgvector lane
142	API operations across 119 paths, with a drift gate that fails the build if the committed OpenAPI schema and the code disagree
30 / 29	database tables / Alembic migrations, applied automatically on boot behind an integrity gate
8	architecture decision records, each with the option that was rejected and why
96	typed configuration settings, every one documented in <code>.env.example</code> with a placeholder

The shape of the backend



Five hard rules, written down before the code and still true:

1. **Only the gateway may import a provider SDK.** Every model call in the product goes through one object that owns routing, budgets, usage, caching and fallback.
2. **All SQL lives in `repositories/`.** Services never write queries; routes never touch repositories.
3. **Every node and edge write goes through the ontology check.** There are no arbitrary edge types.
4. **`usage_records` is append-only.** Nothing updates it and nothing deletes from it.
5. **Every row carries a `user_id`, and every repository method takes one.** Tenant isolation is structural, not a `WHERE` clause someone has to remember.

Nothing calls a model directly

Every model call takes the same six steps

One gateway. No provider SDK is importable anywhere else in the codebase.

1 Typed request

The service builds a `CompletionRequest` carrying a `TaskClass` and a named operation — so every call is attributable.

2 Cache

A content-hash lookup short-circuits repeat work. Re-ingesting the same file costs nothing the second time.

3 Route

Model choice is DATA, not code: `routing_table.py` × the live catalogue, biased hard toward free models.

4 Budget gate

A worst-case projected cost is checked BEFORE the call. Free chains pass at \$0; an over-budget call never leaves.

5 Fallback chain

Candidates are tried in order on 5xx and 429. An EMPTY completion counts as a failure and walks the chain too.

6 Usage recorded

Tokens, cost and the model actually used land in an append-only ledger. Nothing updates it; nothing deletes from it.

Trust boundary — anything a model proposes lands `status = suggested` and is dropped outright if it violates the edge ontology.

The interesting part is what happens when the model misbehaves. An **empty** completion is treated as a failure, not a success — reasoning models will happily burn their entire token budget thinking — and the fallback chain continues. The conceptualizer **drops** any proposal that violates the ontology or references a label it does not know. Everything a model proposes lands as `status = suggested` until a human confirms it. A model can fail to add to your graph. It cannot corrupt it.

Cost is a product feature here rather than telemetry: budgets are checked *before* the call using a worst-case projection, free model chains pass at \$0, and the testing convention is that anything touching routing, budget or usage **must** assert on the recorded usage event or the selected model id.

Local-first is a data model here, not a slogan

Local-first: the files are yours, in your folders

Four stores, one owner each. The database is the source of truth; the file stores are made to converge on it — never the reverse.

ON YOUR DEVICE

The vault — ~/GraphMind PRIMARY

Real, ordinary files — originals, your notes, AI outputs, exports — laid out on disk the way the graph is laid out. Open them in any editor. Back them up any way you like. Uninstall the app and every document is still there.

The index — SQLite, FTS5, embeddings SOURCE OF TRUTH

Documents, nodes, edges and vectors. Deleting a document removes the row and every node, edge, embedding, annotation and pin hanging off it in ONE transaction, committed before a single file is touched.

The blobstore — data/blobs/

App-internal plumbing you are never asked to think about: upload staging, rendered PDFs, cached web snapshots.

OPTIONAL

Google Drive

A secondary mirror of the vault. Off by default, and never the source of truth.

THE RULE, IN ONE LINE

The vault is the user's — report unknown files.

The blobstore is the app's — collect unknown blobs.

A file you put there is never deleted.

An integrity sweep runs hourly, detects orphans across all three stores, auto-repairs the lossless cases and reports the rest.

Deletion is **atomic-or-convergent**: the database commits the removal of a document and every node, edge, embedding, annotation and pin hanging off it in one transaction, *before* a single file is touched. If the file cleanup is then interrupted, all that is left is unreferenced files — and an hourly integrity sweep converges on them, auto-repairing the unambiguous cases and reporting the rest.

The rule that falls out of it is worth memorising: **the vault is the user's, so report unknown files; the blobstore is the app's, so collect unknown blobs**. A file you put in your own vault is never deleted by this software.

What CI actually gates

Nine jobs, path-filtered so a docs change does not pay for a Windows runner:

- **ruff** on source and tests — a hard gate, and currently clean.
- **pytest** — the full backend suite, on Ubuntu, again on **Windows**, and again against **pgvector/pgvector:pg16**.
- **OpenAPI drift** — the schema is regenerated and `git diff --exit-code 'd`. Change a route without regenerating and the build fails.
- **tsc --noEmit** with `strict`, `noUnusedLocals`, `noUnusedParameters`, `noUncheckedIndexedAccess` and `noFallthroughCasesInSwitch` — on the studio and on the generated API client.

- **Playwright**, and it is *blocking*. It was advisory once; the open-document flow regressed three times, so it stopped being advisory.
- `pip-audit` and `npm audit`, plus Dependabot across four ecosystems weekly.
- **Installer smoke build** on every push to main — Inno Setup, artifact uploaded, `if-no-files-found: error`.

Deploys are chained to CI by `workflow_run`, check out **the exact commit that passed** rather than current main, and finish with a five-attempt smoke poll against `/api/ready` that fails the deploy on a bad response.

Things worth pointing at

- **Verified citations.** The RAG stream emits a retrieval frame, then deltas, then a citations frame containing only `[node:<id>]` tags that resolve to real nodes. The ones the model invented are dropped, and counted.
- **A committed retrieval baseline.** 40 queries over a 10-document corpus with hit@1/3/5 and MRR checked in, so a future reranker has to beat a number rather than a vibe.
- **SSRF defence with DNS pinning.** URL ingestion validates the target *and every redirect* against loopback, RFC1918, link-local and the cloud metadata address — then pins the connection to the validated IP through a custom transport, so rebinding does not help either.
- **Hand-rolled JWT verification with an injectable cert source**, specifically so the entire auth path is unit-testable offline against self-signed keys instead of dragging in a grpc dependency tree.
- **Secrets redacted in the log formatter**, not by convention.
- **A test suite that never touches the network** — a scripted fake provider at the provider seam, a real app context on a per-test temporary database, and a hand-assembled minimal PDF fixture so the PDF path is exercised without a PDF-writing dependency.
- **Operational forensics kept in the config file.** The production memory setting carries the actual OOM evidence inline — resident size, exit code, the date — plus the cost delta and the fallback. The graceful-shutdown timeout carries the reason it was raised: the default severed AI streams mid-token.

Honest limits

A brag sheet that only brags is a sales page. These are the gaps, from the project's own documentation:

- **mypy runs in CI but does not gate** — it is `continue-on-error` until the annotations are tightened. Do not read “typed backend” into it.

- **There is no ESLint**, and **no coverage measurement or threshold** anywhere in the repo.
- **The architecture boundaries are enforced by review, not by tooling.** There is no import-linter rule that fails the build on a forbidden import. The ontology *is* mechanically enforced; the layering is not.
- **No CSP or security response headers** are set.
- **Sharing and publishing do not exist yet** beyond per-document public/private for the site API.
- **The Markdown editor is not complete** — no Mermaid, no code syntax highlighting, and fenced blocks render in Preview rather than Live mode. Editing is scoped to note nodes; an uploaded `.md` document is read-only.
- **The Windows installer is unsigned**, so SmartScreen will warn on first run.
- `docs/ARCHITECTURE.md` **has drifted** in two places — it predates the Postgres CI lane and the move off single-machine deployment.

Running it

```
pip install -r requirements.txt      # installs -e ./server[dev]
copy .env.example .env              # then fill in OPENROUTER_API_KEY
python -m graphmind serve           # API at http://127.0.0.1:8000, docs at /docs

cd web/app
npm install && npm run dev           # the studio at http://localhost:5173
```

```
cd server
python -m pytest -q                 # 929 tests, no network required
python -m ruff check src tests      # must stay clean
```

GraphView — a graph-native personal knowledge system.
Your documents, connected, and yours to keep.